Evolution of Java Web Development (Servlets to Spring Boot)

These notes explain how Java web development evolved, why each step was needed, and how frontend was handled at each stage.

1️⃣ Core Java (before web)

What existed

Java used for:

- console programs
- desktop applications (Swing, AWT)

Why it was not enough

- No HTTP support
- No browser interaction
- No concept of web applications

Frontend at that time

- ❌ No frontend
- Output was:
  - console text
  - desktop windows

Why next step was needed

- 👉 Internet and browsers became popular
- 👉 Java needed a way to communicate using HTTP

2️⃣ Servlets (Java meets the web)

What was introduced after the previous step

Servlets (Java EE), which could:

- receive HTTP requests
- send HTTP responses

Why it was needed

Java could finally:

- run on servers
- communicate with browsers

Frontend at that time

HTML was:

- generated directly inside servlets
- written as strings in Java code
- out.println("<html><body>Hello</body></html>");

Problems

- Backend logic and frontend mixed together
- Hard to maintain
- Repetitive and messy code

Why next step was needed

- 👉 Clear separation between business logic and UI was required

---

3️⃣ JSP (JavaServer Pages — first proper frontend separation)

What was introduced after the previous step

JSP, which allowed:

- writing normal HTML
- inserting dynamic data from backend
- JSP runs on the server, not in the browser.

JSP role (VERY IMPORTANT)

JSP's role was to display backend data on the frontend by generating HTML on the server.

- **Backend:** fetches data, applies business logic
- **JSP:** inserts that data into HTML, sends the HTML page to the browser
- 👉 JSP was frontend in purpose, but executed on the backend
- 👉 This is called a server-side frontend technology

Important clarification

- JSP did NOT decide what data to fetch
- JSP did NOT handle business rules
- JSP only rendered what backend prepared

Problems

- Logic leaked into JSP
- Large applications became hard to maintain

Why next step was needed

- 👉 Better backend structure and architecture were required

4️⃣ Spring Framework (backend structure & cleanliness)

What was introduced after the previous step

- Dependency Injection (DI)
- Loose coupling
- Layered architecture (Controller / Service / Repository)

Why it was needed

- Code was tightly coupled
- Difficult to test
- Hard to maintain large applications

Frontend at that time

- Still JSP
- Server-rendered HTML

Limitation (IMPORTANT FOR HISTORY)

No proper web framework yet, meaning:

- No standard request routing
- No controller abstraction
- Developers still managed many servlets manually
- Architecture differed from project to project

Why next step was needed

- 👉 Spring needed a dedicated web framework

5️⃣ Spring Web + Spring MVC (Spring enters web properly)

This is the most important stage for understanding modern Java web development.

When were Spring Web and Spring MVC introduced?

- ❌ Not part of Core Java
- ❌ Not introduced with servlets
- ❌ Not present in early Spring Core
- 👉 They were introduced after Spring Core, when Spring added web support

What Spring Web does

Spring Web provides basic web and HTTP support. It includes:

- HTTP utilities
- Servlet integration
- Web client support
- Low-level request/response handling

Spring Web:

- ❌ does NOT provide controllers
- ❌ does NOT do request routing
- ❌ does NOT expose APIs by itself
- 👉 Spring Web is the foundation layer

What Spring MVC does

Spring MVC is a full web framework built on top of Spring Web. It adds:

- Controllers

- Request mapping
- DispatcherServlet
- Structured request–response flow
- 👉 Spring MVC cannot work alone
- 👉 It always requires Spring Web

Relationship (CRITICAL)

Spring Web can be used alone, and Spring Web + Spring MVC can be used together.

Spring MVC cannot work without Spring Web.

Layered view:

- Spring Framework
  - └── Spring Web (HTTP basics)
    - └── Spring MVC (controllers, routing, APIs)
      - └── Spring Boot (auto-configuration)

Why Spring Web alone was not enough

Because many applications needed:

- routing
- controllers
- a front controller
- structured request handling
- *So Spring introduced Spring MVC on top of Spring Web.*

Types of applications we can build

- **Applications using Spring Web only:**
  - Applications that only make HTTP calls

    - Do NOT receive HTTP requests
    - Example: A Java app calling a weather API and storing the result.
- **Applications using Spring Web + Spring MVC:**
    - Applications that receive HTTP requests
    - Build backend APIs or web servers
    - Example: A backend exposing /login or /users.

Pure Spring vs Spring Boot

- **In pure Spring:**
    - Spring Web / MVC can be used
    - ❌ Configuration is manual (XML / Java configuration required)
- **In Spring Boot:**
    - Same Spring Web / MVC
    - ✅ Auto-configuration
    - Embedded server
    - Faster setup

Spring Boot does NOT replace Spring Web or Spring MVC — it only configures them automatically.

Easy memory rule

- Spring Web only → app talks to others
- Spring Web + Spring MVC → others talk to the app

📘 Transition Note (Important for Understanding the Timeline)

At this stage of the evolution, it is common to feel that later steps appear shorter or less detailed.

This is not because they are less important, but because:

- by this point, the core Java → Spring → web confusion has already been resolved
- the next steps represent architectural shifts, not the introduction of new Java web frameworks

However, for proper learning and revision, these stages must still be explained with the same level of clarity and structure as earlier ones.

6️⃣ Rise of JavaScript Frontend (Frontend–Backend Separation)

What was introduced after the previous step

JavaScript frontend frameworks:

- React
- Angular
- Vue

Why this change was needed

- Server-side pages felt slow
- Full page reloads for UI updates
- Poor user experience
- Mobile apps needed the same backend

What changed architecturally

Frontend and backend became separate applications.

- **Frontend runs:** in browser, in mobile apps
- **Backend handles:** business logic, data processing, security

Frontend at this stage

- Runs completely on client
- Communicates with backend via HTTP APIs
- Receives mostly JSON

Backend at this stage

- No longer returns HTML
- Exposes APIs only
- Uses Spring Web + Spring MVC or Spring Boot

Why this step was important

- 👉 Clear separation of responsibilities
- 👉 One backend supports: web frontend, mobile apps, other systems

---

7️⃣ Spring Boot (Modern Java Backend Standard)

What was introduced after the previous step

- Spring Boot framework

Why Spring Boot was needed

Spring MVC configuration was:

- verbose
- repetitive
- error-prone
- Developers spent more time configuring than coding

What Spring Boot solved

Auto-configuration of:

- Spring Web
- Spring MVC
- DispatcherServlet
- Embedded server (Tomcat)
- Removal of boilerplate setup

What Spring Boot does NOT replace Does NOT replace:

- Spring Framework
- Spring Web
- Spring MVC
- *It uses them internally*

Backend at this stage

- Mostly API-based backends
- JSON responses
- Used by: React apps, mobile apps, other services

Frontend at this stage

Usually separate. Built with:

- React

- Angular
- Mobile UI

Why Spring Boot became the default

- Faster development
- Fewer mistakes
- Production-ready defaults

🔒 **Final Architectural Clarity**

- JavaScript frameworks changed how frontend is built.
- Spring Boot changed how fast backend is built.